

Noise Resistant Encrypted Malicious Traffic Detection Through Kernel-Enhanced Contrastive View Alignment

Meihui Zhong^{ID}, Ting Zhong^{ID}, *Member, IEEE*, Yong Wang^{ID}, Kai Chen^{ID}, *Senior Member, IEEE*,
and Fan Zhou^{ID}, *Member, IEEE*

Abstract—Encrypted Malicious Traffic Detection (EMTD) is challenging due to the limitations of inspecting encrypted content and the scarcity of labeled data. Graph Contrastive Learning (GCL) offers a promising direction by representing traffic as graphs, yet existing approaches struggle to capture global relationships and remain vulnerable to noise. To address these, we propose KerCL, a Kernel-Enhanced Contrastive Learning framework for robust EMTD. KerCL introduces kernel-based graph convolution to capture both local and global traffic dependencies, enhancing the ability to understand the long-range context of encrypted traffic. Furthermore, we propose a dependency-guided graph reconstruction strategy, which restructures adjacency matrices based on learned kernel similarities, ensuring structural integrity while improving diversity in augmented views. To reduce noise-induced semantic drift, we introduce a novel contrastive dependency alignment mechanism. It leverages the Hilbert-Schmidt independence criterion to enforce semantic consistency across augmented views while preserving stability of traffic representations. Experiments on several benchmarks show that KerCL surpasses previous methods in consistency, robustness, and noise tolerance. On the DoHBrw dataset with 50% noise and only 2,000 labels, KerCL improves the F1-score by over 28% compared with the best baseline, showcasing its effectiveness under noisy, label-sparse network conditions.

Index Terms—Encrypted malicious detection, Kernel-based graph convolution, Hilbert-Schmidt independence criterion, graph contrastive learning, self-supervised learning.

I. INTRODUCTION

THE rapid proliferation of encryption protocols like TLS, QUIC, and HTTP/3 has significantly enhanced the privacy and security of online communications. While these

advancements protect user data, they also present significant challenges for Encrypted Malicious Traffic Detection (EMTD) due to the limited visibility into encrypted traffic [1], [2]. Traditional detection methods, such as Deep Packet Inspection (DPI) [3], rely on inspecting packet payloads, which are not accessible in encrypted traffic. This has made the detection of malicious activities concealed within encrypted traffic a critical issue in cybersecurity [4]. Consequently, there is an urgent need for detection methods that can detect malicious traffic without decryption, while maintaining both effective detection and privacy preservation [5].

Recent advancements in Graph Neural Networks (GNNs) offer promising solutions for EMTD by representing network flows as traffic graphs [6], [7], [8], [9]. In these graphs, nodes represent network entities (e.g., IPs, sessions, or flows), and edges capture their interactions. GNNs excel at modeling structural patterns within traffic graphs, which are challenging to capture using sequence-based approaches. This capability makes GNN-based methods promising for detecting malicious encrypted behaviors. However, real-world detection tasks face a key obstacle: high-quality labeled data are scarce [10]. This scarcity has driven the development of Graph Contrastive Learning (GCL)-based methods [11], [12]. GCL is a powerful self-supervised approach, learns discriminative representations by contrasting augmented views of the same graph, reducing reliance on extensive labeled data [13].

Despite significant progress, applying GCL directly to EMTD faces two key challenges – (1) **Limited context**, in traffic data. When modeled as graphs, traffic data within one-hop typically reflect only local interactions between entities. Existing GCL-based methods prioritize optimizing local embeddings, while overlooking global structural dependencies within traffic graphs [14], [15]. As a result, these approaches struggle to capture long-range relationships between network flows. (2) **Susceptibility to noisy artifacts**. Most GCL-based detection approaches rely on heuristic augmentations, such as random node/edge deletion and graph perturbations, to generate contrastive views. These unguided transformations are vulnerable to noise, disrupt semantic consistency, and inject distortions that mislead representation learning [16].

We introduce noise equal in volume to the clean dataset (Figure 1(a)), markedly altering the flow-duration distribution and underscoring its impact on data integrity. Visualization of

Received 4 March 2025; revised 28 August 2025; accepted 19 October 2025; approved by IEEE TRANSACTIONS ON NETWORKING Editor Y. Liu. Date of publication 28 October 2025; date of current version 5 January 2026. This work was supported by the National Natural Science Foundation of China under Grant 62176043, Grant 62072077, and Grant U22A2097. (Corresponding author: Fan Zhou.)

Meihui Zhong and Fan Zhou are with the School of Information and Software Engineering, University of Electronic Science and Technology of China, Chengdu, Sichuan 610054, China (e-mail: zhong99_cs@outlook.com; fan.zhou@uestc.edu.cn).

Ting Zhong is with the School of Information and Software Engineering, University of Electronic Science and Technology of China, Chengdu, Sichuan 610054, China, and also with the Department of Information Security, Kashi Institute of Electronics and Information Industry, Kashgar, Xinjiang 844000, China (e-mail: zhongting@uestc.edu.cn).

Yong Wang is with The Hong Kong University of Science and Technology, Hong Kong, China (e-mail: wangyongjoy@ust.hk).

Kai Chen is with the Computer Science and Engineering, The Hong Kong University of Science and Technology, Hong Kong, China (e-mail: kaichen@cse.ust.hk).

Digital Object Identifier 10.1109/TON.2025.3625606

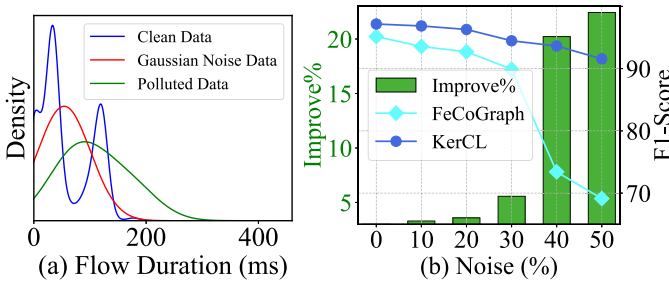


Fig. 1. (a) Adding noise data equivalent to the volume of clean data significantly alters the flow duration distribution, as evidenced by the noticeable shift. (b) FeCoGraph exhibits performance degradation with increasing noise levels, while KerCL demonstrates strong noise robustness, maintaining high F1-scores across the tested range.

the embeddings in the DoHBrw dataset, shown in Figure 1(b), reveals that in an environment where half the data is noise, the FeCoGraph [17] – a label-aware federated GCL method that optimizes intra- and inter-class distances – remains highly sensitive, producing almost indistinguishable features. In contrast, our KerCL effectively mitigates the influence of noise, learning distinct and robust embeddings. It suggests that without proper regularization, contrastive learning can amplify these distortions, causing learned embeddings to deviate from their true distributions. This, in turn, increases errors in similarity evaluations and compromises the model’s ability to maintain stable traffic representations.

To prevent errors or irrelevant information from obscuring semantic, we reconstruct the source graph’s adjacency. We draw inspiration from kernel method, which is a natural approach to capture and quantify semantic relationships between nodes more accurately [18]. This motivates us to focus on reconstructing the graph from a similarity-based kernel matrix. Pairwise similarities aggregate global information and filter minor perturbations, making the reconstructed graph less noise-sensitive [19]. Furthermore, in noisy network environments, augmented samples may include corrupted or adversarial artifacts. Guided by alignment regularization, we introduce a “consensus mechanism” based on dependency-guided alignment, to further reduce noise impact. By aligning the representations of augmented views, the model can emphasize shared, semantically relevant features over noise, as demonstrated in Figure 1(b). The regularization term thus mitigates noise interference, and maintain semantic consistency.

To this end, we propose the **Kernel-Enhanced Contrastive Learning framework (KerCL)**, a novel graph learning approach designed to enhance EMTD. Technically, this models global flow dependencies of through kernels, reconstructs the source graph to generate contrastive views, and introduces a guided alignment mechanism to mitigate noise interference, ultimately improving detection accuracy and robustness. Structurally, it comprises three key components: global dependency modeling, dependency-guided graph reconstruction, and semantic alignment.

Specifically, global dependency modeling first learns a kernel matrix that reflects relations within the original flow graph. Relying on high-order feature propagation, it infers node relations from feature similarity rather than from a

predefined graph structure [18]. This design captures both local and global dependencies, which improves contextual understanding and helps expose subtle malicious patterns spread across sessions. Next, dependency-guided graph reconstruction then uses the kernel similarity matrix to rebuild the adjacency matrix and create augmented views that retain meaningful structural context. By preserving learned flow dependencies, the reconstructed graphs avoid distortions that could otherwise mislead representation learning. Finally, the semantic alignment employs the Hilbert-Schmidt Independence Criterion (HSIC) to implement the dependency-based alignment mechanism. By emphasizing the core features of the source data, this component suppresses irrelevant noise, bolstering the model’s robustness.

The main contributions of this paper are as follows:

- We propose KerCL, a novel Kernel-Enhanced Contrastive Learning framework for robust EMTD, which captures both local and global dependencies in traffic graphs and preserves key semantics. It enhances the detection of subtle malicious patterns under sparse labels and noise conditions.
- We design three integral components to facilitate effective graph learning for EMTD: global dependency modeling, dependency-guided graph reconstruction, and contrastive dependency alignment. These integrate kernel-induced similarity, adjacency restructuring, and HSIC-based alignment, to enhance graph representations in an efficient manner.
- Extensive experiments across three real-world benchmark datasets demonstrate the effectiveness of KerCL. Compared with eight competitive baselines, KerCL achieves average improvements of 5%, 15%, and 18% in accuracy, precision, and F1-score, respectively, on the TONIoT dataset, even under challenging conditions with 50% noise and limited labeled samples.

Roadmap. The remainder of this paper is structured as follows: Section II reviews the related work on EMTD. Section III details the proposed KerCL framework. In Section IV, we reports extensive experiments, including overall effectiveness, robustness, ablation studies, and complexity analysis. Section V concludes the study and outlines future work.

II. RELATED WORK

A. Encrypted Malicious Traffic Detection

EMTD has become central to network defense as encryption protocols proliferate. Existing approaches fall into three broad classes.

(1) **Rule-based Methods.** These methods rely on constructing blacklists of IP addresses, certificates, or ports associated with known attackers [20], [21]. Although fast and easy to deploy, they are inherently static; adversaries can evade them by frequently changing observable features, and their effectiveness therefore decays in dynamic environments.

(2) **Statistics-based Methods.** These approaches learn from flow-level statistics such as packet size, inter-arrival time, and session duration. For instance, XGBoost [22] uses sparse sensing and weighted quantile sketches to model high-dimensional

traffic features. Similarly, AppScanner [23] develops a framework for application fingerprinting, incorporating strategies to identify and mitigate ambiguous traffic effects. However, such methods are vulnerable to traffic variation and noise, which lead to elevated false-positive and false-negative rates when data are sparse or noisy.

(3) **Deep Learning-based Methods.** These methods offer powerful alternatives by learning complex representations from raw network data [24], [25]. They can be further classified by feature extraction strategies: *Packet-level models*, such as Deep Fingerprinting [26] treats website trace as a sequence of packets, leveraging sequential learning method to classify traffic flows using raw packet data. FS-Net [27] similarly processes raw packet data, mining potential sequence features from streams, and incorporating the reconstruction mechanism to improve the features effectiveness. *Graph-based methods*, including GraphDApp [24], TrafficGCN [28], ST-Graph [29] and EC-GCN [30], transform sequences into graphs. They represent dependencies among entities (e.g., IPs, sessions, or flows) for effective representation learning. And *image-based methods*, such as TAM-CNN [31] and ATVTSC [25], treat traffic data as images, leveraging CNNs for EMTD.

B. Semi-Supervised Learning

Semi-Supervised Learning (SSL) has gained traction as a means of extracting supervisory signals from unlabeled data, offering a promising direction for tasks like EMTD. SSL is particularly valuable in graph-based tasks, where large amounts of unlabeled data can be utilized to learn effective representations. Recent studies in EMTD have explored several SSL frameworks, categorized as follows:

(1) **Data-Level Augmentation.** Models such as TrafCL [32] randomly drops packets to generate augmented views. MTC-MAE [33] constructs a unsupervised pretext task with a random masking strategy to get augmentation views. FeCo-Graph [17], a label-aware federated GCL framework, enhances flow embedding discriminability by optimizing intra-class and inter-class distances; These methods effectively exploit data variability but risk semantic loss under noisy conditions due to unguided augmentations, limiting their robustness in EMTD's label-scarce, noisy environments.

(2) **Representation-Level Augmentation.** Some methods inject perturbations into inputs to regularize representation distributions, generating semantically diverse views. For instance, TCG-IDS [34] proposes three graph contrastive strategies: temporal contrasting to capture node evolution over time, asymmetric contrasting to model relationships between nodes, and random masking of node or edge features during training to force the model to learn from residual data; While boosting representation flexibility, these heuristic perturbations can amplify noise distortions, exacerbating semantic inconsistency in noisy encrypted traffic, as highlighted in our earlier analysis.

(3) **Local-Global Augmentations.** CoMDet [35] introduces an inter-modal contrastive learning approach, enhancing feature representations by maximizing mutual information across modalities. PeerG [36] employs graph contrastive learning to optimize feature extraction for P2P botnet detection, using maximum mutual information to measure local and global

information in communication graphs, thereby identifying P2P botnet nodes. Although they capture global aspects, their modality-specific or task-specific focus limits comprehensive modeling of flow dependencies in noisy, encrypted traffic, falling short of addressing EMTD's structural complexity.

In contrast, our KerCL learn the nonlinear dependencies of augmented representations in high-dimensional space, employs dependency-guided reconstruction for consistent augmented views, and applies HSIC-based alignment for noise-resistant, uniform representations.

III. METHODOLOGY

This section elaborates on the framework of KerCL in detail. Fig. 2 provides an overview of KerCL, which integrates global dependency modeling, graph reconstruction, and semantic alignment. The model is designed to capture both local and global dependencies in traffic graphs while generating semantically consistent augmentations for robust graph learning. This combination ensures effective and generalizable detection of malicious traffic, even in noisy and label-scarce environments.

A. Task Definition

Encrypted network traffic comprises data packets exchanged between hosts within a network. This traffic can be analyzed at multiple granularities, from individual packets to aggregated entities such as flows, sessions, or channels. We adopt a flow-based setting: a flow is a sequence of packets sharing a common 4-tuple: source IP, destination IP, source port, destination port.

We model encrypted network traffic as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{A}, \mathbf{H})$. Nodes $v_i \in \mathcal{V}$ represent hosts identified by IP addresses. Edges $\mathcal{E}$ are established using heuristics such as flows sharing the same destination IP address and port. The binary adjacency matrix $\mathbf{A} \in \{0, 1\}^{n \times n}$ encodes these connections, where $\mathbf{A}_{ij} = 1$ indicates an edge and $\mathbf{A}_{ij} = 0$ otherwise. Each node is associated with a feature vector in the initial node-feature matrix constructed from raw per-flow/session features $\mathbf{H} \in \mathbb{R}^{n \times d}$, where each row contains a d -dimensional feature vector, and this matrix serves as the input to the propagation stage. Feature fields include common flow statistics such as packet counts, flow duration, and inter-packet intervals. We consider a C -class classification problem with label set $\mathcal{Y} = \{1, \dots, C\}$. For each labeled node v_i , $y_i \in \mathcal{Y}$ denotes its class, and we use $c \in \mathcal{Y}$ to index classes.

Building on prior graph-based approaches to EMTD [12], [37], we implement a semi-supervised transductive learning framework. Given $\mathcal{G}$ with n_l labeled nodes, our goal is to learn $f(\cdot)$ to predict the label distribution $\hat{y}_t$ for an unlabeled node v_t by leveraging both labeled and unlabeled nodes.

B. Kernel Enhanced Global Dependency Modeling

In network traffic analysis, capturing both local and global dependencies is essential for detecting malicious behaviors hidden in encrypted flows. Local dependencies refer to direct communication between two IP addresses, such as traffic exchanged within a single session or flow. Global dependencies capture broader relationships across multiple, potentially

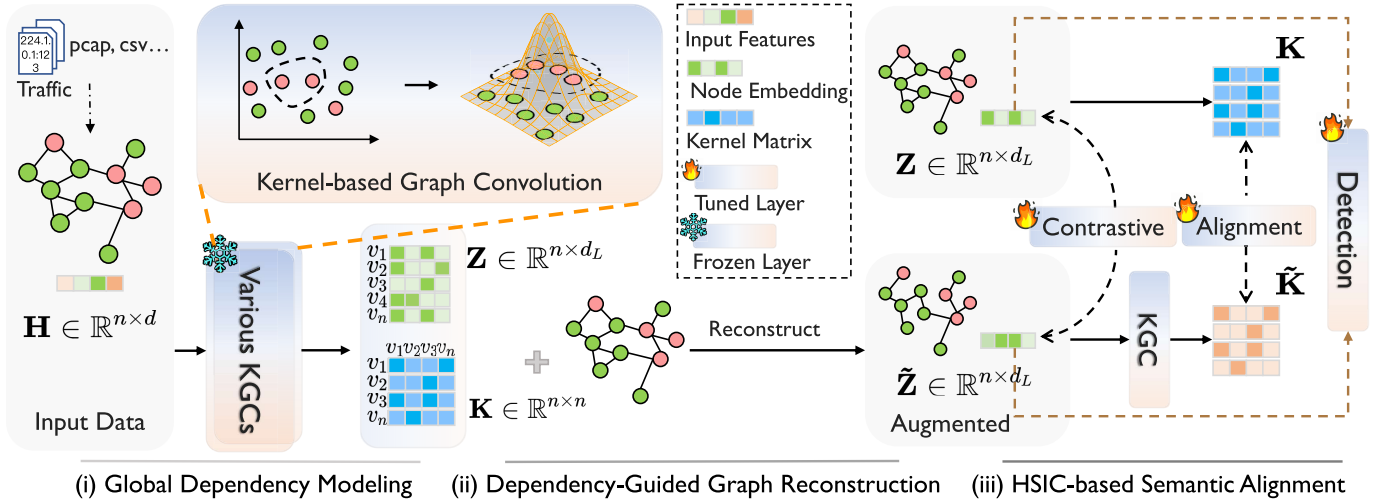


Fig. 2. The architecture of KerCL. (i) Global Dependency Modeling, which captures both local and global dependencies in the traffic graph through various Kernel-based Graph Convolutions (KGCs); (ii) Dependency-Guided Graph Reconstruction, which generates semantically consistent augmented graphs using a learned kernel matrix; (iii) HSIC-based Semantic Alignment, which aligns original and augmented embeddings ($\mathbf{Z}$ and $\tilde{\mathbf{Z}}$) using the Hilbert-Schmidt Independence Criterion (HSIC) and contrastive learning. The final detection module objective combines cross-entropy and alignment losses.

distant nodes, such as flows originating from a malicious IP over different sessions. Traditional GNNs model capture these relationships by aggregating information from neighboring nodes based on graph adjacency, a structure-only approach that relies solely on the graph's topological connections without incorporating semantic similarities. This structure-only approach has two key limitations: (1) stacking multiple layers to capture long-range dependencies often lead models suffer from over-smoothing, causing node embeddings to lose distinctiveness, and (2) reliance on adjacency matrices alone overlooks semantic similarities between flows based on shared behavioral patterns, hindering detection of non-adjacent flows with similar features.

(1) Kernel-based Graph Convolution.

To overcome these limitations, we propose a kernel-enhanced global dependency modeling method that integrates structural graph information with semantic dependencies through the Kernel-based Graph Convolution (KGC). This method implicitly encodes pairwise relationships by mapping node features into a high-dimensional space using kernel functions. And effectively balances the representation of local and global dependencies without requiring deeper architectures, ensuring scalability and resilience to noise.

a) Kernel Mapping: A core component of our method is the kernel matrix $\mathbf{K}$, which encodes pairwise relationships between nodes by implicitly mapping their features into a high-dimensional space. In network security, these relationships capture the behavioral similarity between hosts, a critical signal for detecting coordinated malicious activities which generate statistically similar traffic patterns. This projection is designed to improve class separability, that is, to what extent to which learned representations for different classes are distinct and easily distinguishable. By leveraging kernel-induced similarities, this mapping aims to increase the margins between classes while reducing the variance within each class, making the subsequent classification task more effective.

Formally, the mapping is defined as:

$$\text{MAPPING}(\mathbf{H}) := \psi(\mathbf{H}) = [\psi(\mathbf{h}_1), \psi(\mathbf{h}_2), \dots, \psi(\mathbf{h}_n)]^\top, \quad (1)$$

where $\psi: \mathbb{R}^d \rightarrow \mathbb{R}^D$ is a feature mapping and $d \leq D$.

Direct computation of explicitly designing the mapping function $\psi(\cdot)$, is computationally prohibitive, particularly for large or infinite D . Instead, we utilize a kernel function $k(\cdot, \cdot)$, which implicitly computes the inner product in the mapped space: $k(\mathbf{h}, \mathbf{h}') = \langle \psi(\mathbf{h}), \psi(\mathbf{h}') \rangle$. Hence, the kernel matrix $\mathbf{K}$ for the node set $\mathbf{H}$ is constructed as:

$$\mathbf{K} = \begin{bmatrix} k(\mathbf{h}_1, \mathbf{h}_1) & \cdots & k(\mathbf{h}_1, \mathbf{h}_n) \\ \vdots & \ddots & \vdots \\ k(\mathbf{h}_n, \mathbf{h}_1) & \cdots & k(\mathbf{h}_n, \mathbf{h}_n) \end{bmatrix}, \quad (2)$$

where each kernel corresponds to an implicit feature map. For instance, a polynomial kernel $k(\mathbf{h}, \mathbf{h}') = (\mathbf{h}^\top \mathbf{h}' + c)^d$ corresponds to a d -order polynomial feature map, while a Gaussian kernel $k(\mathbf{h}, \mathbf{h}') = \exp(-\|\mathbf{h} - \mathbf{h}'\|^2 / (2\sigma^2))$ represents an infinite-dimensional mapping, with σ controlling sensitivity to feature differences.

b) Kernel Propagation: Our KGC's propagation mechanism updates node embeddings $\mathbf{H}^{(l)}$ iteratively on the graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{A}, \mathbf{H})$, where $\mathbf{H} = [\mathbf{h}_1; \dots; \mathbf{h}_n] \in \mathbb{R}^{n \times d}$ denotes the initial raw node-feature matrix constructed from the original per-flow/session statistics.

This leverages high-dimensional feature mappings to encode nonlinear relationships, expressed as:

$$\bar{\mathbf{H}}^{(l+1)} = \text{AGGREGATE}(\mathcal{G}, \mathbf{H}^{(l)}) = \hat{\mathbf{A}}^q \mathbf{H}^{(l)}, \quad (3)$$

$$\mathbf{H}^{(l+1)} = \psi_{(l)}(\bar{\mathbf{H}}^{(l+1)}), \quad (4)$$

where $\hat{\mathbf{A}} = \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$ is the normalized adjacency matrix, q controls the propagation range and enables multi-hop aggregation, and l denotes the layer index. In particular, we define the initial kernel matrix $\mathbf{K}^{(0)} = \mathbf{H} \mathbf{H}^\top$, where $\mathbf{H}$ is the initial input feature matrix.

Obviously, it is easy to observe that the kernel matrix for the updated embeddings can be expressed as $\mathbf{K}^{(l+1)} = \mathbf{H}^{(l+1)}\mathbf{H}^{(l+1)\top}$. However, directly computing $\mathbf{H}^{(l+1)}$ is impractical due to its implicit nature. Instead, we aim to derive $\mathbf{K}^{(l+1)}$ directly from $\mathbf{K}^{(l)}$, thereby bypassing the need to explicit embedding calculations. To achieve this, we establish a relationship between the kernel matrices. For a Gaussian kernel, the updated kernel matrix $\mathbf{K}^{(l+1)}$ is defined as:

$$\mathbf{K}^{(l+1)} = \exp\left(-\frac{\Delta_{ij}^{(l+1)}}{2\gamma^2}\right), (i, j) \in [n] \times [n], \quad (5)$$

where $\Delta_{ij}^{(l+1)} = \|\bar{\mathbf{h}}_i^{(l+1)} - \bar{\mathbf{h}}_j^{(l+1)}\|^2$ is an element of the squared distance matrix $\Delta^{(l+1)}$ for $\bar{\mathbf{H}}^{(l+1)}$, and γ controls sensitivity to feature differences.

Through a series of derivations, we have:

$$\Delta^{(l+1)} = (\hat{\mathbf{A}}^q)_i \mathbf{K}^{(l)} (\hat{\mathbf{A}}^q)_i^\top - 2(\hat{\mathbf{A}}^q)_i \mathbf{K}^{(l)} (\hat{\mathbf{A}}^q)_j^\top + (\hat{\mathbf{A}}^q)_j \mathbf{K}^{(l)} (\hat{\mathbf{A}}^q)_j^\top, \quad (6)$$

where $\hat{\mathbf{A}}^q$ is the normalized adjacency matrix raised to the q -th power, enabling multi-hop propagation. For convenience, this can be written in a more compact matrix form:

$$\Delta^{(l+1)} = \mathbf{d}_{\bar{\mathbf{K}}^{(l+1)}} \mathbf{1}_n^\top + \mathbf{1}_n \mathbf{d}_{\bar{\mathbf{K}}^{(l+1)}}^\top - 2\bar{\mathbf{K}}^{(l+1)}, \quad (7)$$

where

$$\mathbf{d}_{\bar{\mathbf{K}}} = [\bar{K}_{11}, \bar{K}_{22}, \dots, \bar{K}_{nn}]^\top, \quad (8)$$

$$\bar{\mathbf{K}}^{(l+1)} = \hat{\mathbf{A}}^q \mathbf{K}^{(l)} (\hat{\mathbf{A}}^q)^\top. \quad (9)$$

This propagation step enriches the kernel matrix to include structural information from the graph while preserving node-to-node similarity relationships. Consequently, the updated kernel matrix can be computed as:

$$\mathbf{K}^{(l+1)} = \exp\left(-\frac{\mathbf{d}_{\bar{\mathbf{K}}^{(l+1)}} \mathbf{1}_n^\top + \mathbf{1}_n \mathbf{d}_{\bar{\mathbf{K}}^{(l+1)}}^\top - 2\bar{\mathbf{K}}^{(l+1)}}{2\gamma_{(l+1)}^2}\right). \quad (10)$$

Note that now this formulation avoids the explicit representation from $\psi_{(l)}(\bar{\mathbf{H}}^{(l+1)})$, but is directly obtained from $\mathbf{K}^{(l)}$. Our kernel-enhanced approach effectively addresses the over-smoothing problem of traditional GNNs by avoiding deep architectures, while learning rich semantic information from encrypted traffic graphs to capture both structural and behavioral patterns, enabling robust detection of subtle malicious behaviors in EMTD.

C. Dependency-Guided Graph Reconstruction

Existing augmentation techniques for graph contrastive learning, such as random node or edge removal and feature masking, are commonly used to enhance data diversity. However, these heuristic approaches often disrupt the intrinsic semantic structure of the graph, leading to suboptimal learning outcomes. To address these limitations, we propose a dependency-guided graph reconstruction mechanism, which leverages the kernel matrix $\mathbf{K}^{(L)}$ (abbreviated as $\mathbf{K}$ hereafter) to generate an augmented adjacency matrix. This process generates an augmented graph that is structurally different from

the original but preserves the behavioral relationships between nodes, which is crucial for effective contrastive learning.

The reconstruction process begins with the kernel matrix $\mathbf{K}$, and we construct the augmented adjacency matrix $\tilde{\mathbf{A}}$ as:

$$\tilde{\mathbf{A}}_{ij} = \begin{cases} 1, & \text{if } j \neq i \text{ and } \mathbf{K}_{ij} \geq c_i, \\ 0, & \text{otherwise,} \end{cases} \quad (11)$$

where c_i is the threshold corresponding to the k -th largest similarity score in each row i . This formulation ensures that only the top- k most semantically similar neighbors for each node are retained, preserving critical relationships while reducing noise-induced distortions.

To further enhance diversity, we introduce Gaussian noise injection in the reconstructed graph. This introduces small perturbations to the edge weights in $\tilde{\mathbf{A}}_{ij}$, simulating minor variations in data distribution without compromising the overall semantic consistency. Formally, the noise is applied as:

$$\tilde{\mathbf{A}}_{ij} \leftarrow \tilde{\mathbf{A}}_{ij} + \mathcal{N}(0, \sigma^2), \quad (12)$$

where $\mathcal{N}(0, \sigma^2)$ represents Gaussian noise with zero mean and variance σ^2 .

Once the augmented adjacency matrix is constructed, we apply normalization and sparsification to ensure computational efficiency and numerical stability. Normalization is performed as:

$$\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}}, \quad (13)$$

where $\tilde{\mathbf{D}}$ is the degree matrix corresponding to $\tilde{\mathbf{A}}$. This step preserves key semantic information while optimizing the graph for graph learning tasks.

By leveraging the kernel matrix to guide graph reconstruction, our KerCL effectively balances semantic preservation with meaningful diversity. This dependency-guided mechanism mitigates the semantic disruption typically associated with random augmentation methods. As a result, the graph reconstruction mechanism generates high-quality augmented views that meet the requirements of contrastive learning and enhance model generalization.

D. HSIC-Based Semantic Alignment

While the kernel-based graph convolution provides embeddings with good initial separability, we introduce a *self-supervised semantic alignment* step to make these representations more robust to noise. This improved robustness, in turn, further enhances the final class separability. To achieve this, we design a graph contrastive learning method that incorporates semantic alignment through the Hilbert-Schmidt Independence Criterion (HSIC) [38]. This method ensures that the representations from the original graph and its augmented view remain consistent with the underlying semantics, addressing challenges such as semantic drift and representation collapse, which are often encountered in self-supervised learning on graphs.

(1) Contrastive Learning with Instance-Level Alignment.

At the core, our method begins with a contrastive loss term designed to promote instance-level consistency between embeddings of the original graph and its augmented counterpart. Let $\mathbf{Z}$ and $\tilde{\mathbf{Z}}$ represent the node embeddings learned

from the original and augmented graphs, respectively. The contrastive loss is formulated as:

$$\mathcal{L}_c = -\frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} \log \frac{\exp(\text{sim}(\tilde{\mathbf{Z}}_i, \tilde{\mathbf{Z}}_i)/\tau)}{\sum_{j \in \mathcal{V}} \exp(\text{sim}(\mathbf{Z}_i, \tilde{\mathbf{Z}}_j)/\tau)}, \quad (14)$$

where $\text{sim}(\cdot, \cdot)$ denotes a similarity measure (e.g., cosine similarity), τ is the temperature parameter, $\mathbf{Z}_i$ is the embedding of node i in the original graph, and $\tilde{\mathbf{Z}}_i$ is the corresponding embedding in the augmented graph. This loss aligns individual node embeddings across views while ensuring separability between embeddings of different nodes.

(2) Semantic Alignment via HSIC.

While the contrastive loss ensures instance-level alignment, it does not enforce the global consistency across the graph's semantic structure. To address this, we introduce a semantic alignment mechanism based on HSIC. HSIC could measures statistical dependency between kernel matrices $\mathbf{K}$ and $\tilde{\mathbf{K}}$, which are derived from the embeddings $\mathbf{Z}$ and $\tilde{\mathbf{Z}}$, respectively. Similarly, through KGC" we can calculate $\tilde{\mathbf{K}}$ in the following matrix form:

$$\tilde{\mathbf{K}} = \exp \left(-\frac{\tilde{\Delta}}{2\gamma^2} \right). \quad (15)$$

where $\tilde{\Delta}$ is the squared distance matrix of $\tilde{\mathbf{H}}^{(L+1)}$ for the augmented graph, and γ matches the definition given in Eq. 5.

HSIC quantifies the dependency between original and augmented representations, estimated as:

$$\begin{aligned} \widehat{\text{HSIC}}(\mathbf{Z}, \tilde{\mathbf{Z}}) &= \|\mathbb{E}[\mathbf{K}\tilde{\mathbf{K}}^\top] - \mathbb{E}[\mathbf{K}]\mathbb{E}[\tilde{\mathbf{K}}]^\top\|_{\text{HS}}^2 \\ &= \frac{\text{tr}(\mathbf{Q}\mathbf{K}\mathbf{Q}\tilde{\mathbf{K}}\mathbf{Q})}{(n-1)^2}, \end{aligned} \quad (16)$$

where $\mathbf{Q} = \mathbf{I}_n - \frac{1}{n}\mathbf{1}_n\mathbf{1}_n^\top$ is the centering matrix, and $\text{tr}(\cdot)$ denotes the matrix trace.

In addition to enforcing embedding uniformity and stability, we incorporate regularization terms to penalize high-variance representations. Specifically, we include self-dependence terms $\widehat{\text{HSIC}}(\mathbf{Z}, \mathbf{Z})$ and $\widehat{\text{HSIC}}(\tilde{\mathbf{Z}}, \tilde{\mathbf{Z}})$, which minimize variance within the original and augmented graph embeddings, respectively. To balance these objectives, we define the overall kernel dependence maximization loss as:

$$\begin{aligned} \mathcal{L}_s &= -\widehat{\text{HSIC}}(\mathbf{Z}, \tilde{\mathbf{Z}}) \\ &+ \xi_1 \cdot \widehat{\text{HSIC}}(\mathbf{Z}, \mathbf{Z})^{-\frac{1}{2}} + \xi_2 \cdot \widehat{\text{HSIC}}(\tilde{\mathbf{Z}}, \tilde{\mathbf{Z}})^{-\frac{1}{2}}, \end{aligned} \quad (17)$$

where ξ_1 and ξ_2 control the importance of the regularization terms. These terms ensure that embeddings remain well-distributed and stable during training.

(3) Integration with Contrastive Learning.

To combine instance-level alignment with global semantic consistency, we integrate $\mathcal{L}_s$ with the contrastive loss $\mathcal{L}_c$, forming the final contrastive training objective:

$$\mathcal{L}_{\text{HSICSA}} = \lambda_c \cdot \mathcal{L}_c + \lambda_s \cdot \mathcal{L}_s, \quad (18)$$

where λ_c and λ_s are hyperparameters controlling the contributions of the contrastive and kernel dependence terms, respectively. This integrated loss enhances KerCL's robustness, mitigating semantic drift and noise-induced distortions.

Algorithm 1 The training algorithm of KerCL.

Input: Adjacency matrix $\mathbf{A}$; Feature matrix $\mathbf{H}$;
Ground truth labels $\mathbf{Y}$; Hyperparameters $\gamma, k,$
 τ, ξ_1, ξ_2

Output: Model parameters Θ

- 1 **Initialize** parameters in Θ ;
/* Flow Dependency Modeling */
- 2 Compute kernel matrix $\mathbf{K} \leftarrow \text{propagation}(\mathbf{A}, \mathbf{H}, \gamma)$ via Eq. 10;
/* Graph Reconstruction */
- 3 Compute reconstructed adjacency matrix
 $\hat{\mathbf{A}} \leftarrow \text{kNN}(\mathbf{K}, k)$ via Eq. 11;
- 4 Generate noisy matrix $\hat{\mathbf{A}}$ via Eq. 12 and Eq. 13;
/* Semantic Alignment */
- 5 Compute HSIC within each view via Eq. 16;
/* **Training** */
- 6 **while not converged do**
- 7 Compute overall loss $\mathcal{L}$ from $\mathcal{L}_{\text{CE}}, \mathcal{L}_{\text{CE}_A}$, and $\mathcal{L}_{\text{HSICSA}}$ via Eq. 21;
- 8 Update Θ via gradient descent;
- 9 **end**

E. Training Objective

Our downstream task is C -class node classification. Binary benign malicious classification corresponds to $C = 2$, while other datasets use $C > 2$. To perform this classification, we attach a lightweight classifier head to the final embeddings ($\mathbf{Z}$ and $\tilde{\mathbf{Z}}$) from the original and augmented graphs. This classifier is an MLP that maps the embeddings to predicted label distributions $\hat{\mathbf{Y}}$ and $\hat{\tilde{\mathbf{Y}}}$, respectively:

$$\hat{\mathbf{Y}} = \text{MLP}(\mathbf{Z}), \quad \hat{\tilde{\mathbf{Y}}} = \text{MLP}(\tilde{\mathbf{Z}}), \quad (19)$$

where $\hat{\mathbf{Y}} \in \mathbb{R}^{N \times C}$ and $\hat{\tilde{\mathbf{Y}}} \in \mathbb{R}^{N \times C}$ represent the predicted probability distributions over C classes for the nodes in the original and augmented graphs, respectively.

The training process leverages ground truth labels $\mathbf{Y} \in \mathbb{R}^{N \times C}$ for a subset of nodes $\mathcal{V}_{\text{train}} \subset \mathcal{V}$. For the original graph embeddings, we calculate the cross-entropy loss $\mathcal{L}_{\text{CE}}$ as:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|\mathcal{V}_{\text{train}}|} \sum_{i \in \mathcal{V}_{\text{train}}} \sum_{c=1}^C \mathbf{Y}_{i,c} \log \hat{\mathbf{Y}}_{i,c}, \quad (20)$$

where $\mathbf{Y}_{i,c}$ is the ground truth label for node i in class c , and $\hat{\mathbf{Y}}_{i,c}$ is the predicted probability of node i belonging to class c . Similarly, for the augmented graph embeddings, we compute the augmented cross-entropy loss $\mathcal{L}_{\text{CE}_A}$.

By jointly optimizing these loss, the model learns embeddings that are predictive of node labels and robust to structural and feature variations from graph augmentations. Finally, the joint loss function is defined as follows:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_e \cdot \mathcal{L}_{\text{CE}_A} + \mathcal{L}_{\text{HSICSA}}, \quad (21)$$

where λ_e is a hyperparameter controlling the contribution of the augmented view's classification loss. Algorithm 1 provides the pseudo algorithm of the whole process of our method. We compute the classification cross-entropy only on labeled

nodes, while both the contrastive objective and the HSIC view-alignment objective are computed on all nodes and require no labels.

IV. EVALUATION

We conduct experiments to address the following Research Questions (RQs):

RQ1: How is the efficiency of KerCL (Section IV-B)?

RQ2: How is the robustness of KerCL (Section IV-C)?

RQ3: How is the usefulness of each component (Section IV-D)?

RQ4: To what extent can changes in hyper-parameters affect the effectiveness of KerCL (Section IV-E)?

A. Experimental Setup

Datasets. We benchmark our proposed KerCL against the following approaches.

(i) *TONIoT* [39]: This dataset represents a new generation of Internet of Things (IoT) and Industrial Internet of Things (IIoT) traffic. It comprises 796,380 normal and 21,542,641 malicious instances with 10 distinct types, encompassing 39 statistical features;

(ii) *DoHBrw* [40]: This dataset is designed for the analysis, testing, and evaluation of DNS over HTTPS (DoH) encrypted traffic. This dataset offers 19,807 normal and 354,996 malicious instances, each characterized by 28 statistical features;

(iii) *CICIDS* [41]: This dataset contains both normal communication patterns and 14 distinct attack scenarios, with 931,873 normal instances and 18,028 malicious instances. Each sample is described by 70 features, extracted using the CICFlowMeter-V3 [42].

Baselines. To benchmark our KerCL, we choose the following approaches for evaluation:

(i) Supervised based EMTD, including

MTSecurity [7]. It integrates raw byte-level features (via Transformers) and graph-based traffic interaction features (via GNNs) to enhance encrypted traffic representations, capturing sequence and network patterns despite payload invisibility.

(ii) Unsupervised based EMTD, including

Kitsune [2]. It detects anomalies using unsupervised learning on raw traffic, adapted here for supervised classification; *ContraMTD* [12]. It models both sequence and topological relationships in host interaction graphs using proposed SEC and DE-GAT mechanisms. It employs contrastive learning with vertical and horizontal views of traffic behavior. This enables comprehensive modeling of normal network behavior across different views.

(iii) Pre-train based EMTD, including:

ET-BERT [43]. It utilizes the raw traffic data of the first n packets of flow as their input. It transforms the datagram of bytes to a language-like token and uses BERT pre-training methods for learning the contextualized datagram presentation from large unlabeled datasets. After that, it uses fine-tuning as its tuning method to transfer the well-learned presentation to downstream tasks;

Flow-MAE [44]. It improves upon ET-BERT framework by adopting an asymmetric encoder-decoder architecture, which is similar to masked autoencoder (MAE). This architecture

TABLE I
DATASET STATISTICS

Dataset	# Nodes	# Edges	# Features	# Classes
TONIoT	1,000,000	9,287,986	39	10
Exp. Subset	10,000	2,360	39	10
DoHBrw	269,643	582,246	28	2
Exp. Subset	2,000	802	28	2
CICIDS	949,901	7,947,136	70	15
Exp. Subset	15,000	271,584	70	15

¹ **Exp. Subset** indicates the settings used in the experiment.

generates representations based on byte streams and uses a lightweight decoder to reconstruct the masked bytes;

YaTC [45]. It utilizes the raw traffic data of the first n packets of flow as their input. It also uses the MAE paradigm as its pre-training method by reconstructing the masked traffic. After that, it uses fine-tuning as its tuning method for downstream tasks as well;

TrafCL [32]. It enhances session-level data by creating augmented, incomplete views of the sessions. These incomplete views are then input into a contrastive learning framework, enabling better representation learning from partial data;

FeCoGraph [17]. It is a label-aware federated GCL framework optimizing intra-class and inter-class distances for flow embedding discriminability.

Metrics. Our model aims to classify encrypted traffic into normal and various malicious types. Given the inherent class imbalance in malicious and normal traffic, we evaluate our KerCL and other using accuracy (AC), precision (PR), and F1-score (F1).

Testbeds. Our experiments are conducted on a platform with an Intel i5-14600KF 3.50 GHz CPU, an NVIDIA GeForce GTX 4060 Ti GPU, and 32GB of system memory. We implemented graph-based encoders for baselines models using the PyTorch Geometric (PyG) library. Additionally, since Kitsune cannot handle high-rate traffic, we evaluate it with offline experiments on the same testbed.

Implementation Details. To further evaluate the robustness of KerCL, all experiments were conducted under a noise level of 0.3 to simulate real-world noisy environments. Unless otherwise specified, Gaussian noise was applied to the original data, and the number of training nodes was limited, with 1,000 labeled samples per category used. The Adam optimizer is employed for model training. Early stopping was employed, halting training if no parameter improvement was observed for 50 consecutive epochs.

B. Overall Performance Evaluation (Q1)

To validate the generalizability of the proposed KerCL, we conducted extensive evaluations on three benchmark datasets representing diverse traffic patterns and malicious behaviors. The results, shown in Table II, demonstrate that KerCL consistently achieves SOTA performance across key evaluation metrics. These highlight KerCL's ability to effectively learn robust and discriminative representations of encrypted traffic, even in challenging conditions with limited labeled samples.

TABLE II

PERFORMANCE COMPARISON OF KERCL WITH BASELINES UNDER A ENVIRONMENT OF EXPERIMENTAL SETUP ACROSS THREE DATASETS. THE BEST AND RUNNER-UP RESULTS ARE HIGHLIGHTED IN **BOLD** AND UNDERLINED, RESPECTIVELY

Dataset	TONIoT			DoHBrw			CICIDS		
	ACC	PR	F1	ACC	PR	F1	ACC	PR	F1
MTSecurity	74.61 (± 0.65)	74.68 (± 0.64)	74.61 (± 0.67)	83.86 (± 0.60)	83.86 (± 0.60)	83.86 (± 0.60)	83.85 (± 0.17)	83.35 (± 0.17)	52.16 (± 0.40)
Kitsune	66.85 (± 0.71)	67.27 (± 0.73)	66.75 (± 0.72)	73.26 (± 1.09)	73.33 (± 1.09)	73.10 (± 1.09)	72.04 (± 0.53)	60.24 (± 0.52)	66.15 (± 0.55)
ContraMTD	73.60 (± 0.83)	73.90 (± 0.76)	73.57 (± 0.89)	77.88 (± 1.25)	78.60 (± 1.21)	74.03 (± 1.23)	72.13 (± 0.73)	85.73 (± 0.66)	71.77 (± 0.71)
ET-BERT	81.91 (± 0.51)	82.47 (± 0.53)	81.68 (± 0.53)	91.68 (± 0.81)	91.68 (± 0.81)	91.68 (± 0.81)	84.00 (± 0.31)	84.57 (± 0.30)	83.79 (± 0.32)
Flow-MAE	80.35 (± 0.42)	81.07 (± 0.42)	79.81 (± 0.42)	86.50 (± 0.88)	86.50 (± 0.88)	86.50 (± 0.97)	83.99 (± 0.45)	83.50 (± 0.36)	83.77 (± 0.40)
YaTC	84.31 (± 0.32)	77.88 (± 0.30)	75.49 (± 0.33)	<u>94.01 (± 0.96)</u>	<u>94.01 (± 0.96)</u>	<u>94.00 (± 0.97)</u>	<u>87.57 (± 0.19)</u>	<u>87.25 (± 0.21)</u>	<u>86.76 (± 0.19)</u>
TrafCL	82.83 (± 0.65)	<u>83.48 (± 0.62)</u>	<u>82.78 (± 0.65)</u>	87.70 (± 0.85)	87.70 (± 0.76)	87.37 (± 0.77)	84.10 (± 0.41)	84.49 (± 0.47)	83.82 (± 0.43)
FeCoGraph	<u>84.59 (± 0.44)</u>	71.98 (± 0.48)	70.42 (± 0.45)	92.98 (± 0.81)	93.01 (± 0.77)	92.71 (± 0.79)	84.12 (± 0.30)	83.87 (± 0.38)	79.34 (± 0.26)
KerCL (Our)	88.23 (± 0.22)	88.56 (± 0.20)	88.23 (± 0.22)	95.83 (± 0.36)	95.58 (± 0.36)	95.57 (± 0.37)	90.13 (± 0.21)	90.49 (± 0.42)	90.06 (± 0.21)

Supervised methods, such as MTSecurity, heavily rely on large amounts of labeled data for training, a limitation in real-world scenarios where annotated datasets are either scarce or expensive to obtain. While these methods perform well on datasets with abundant labels, their effectiveness diminishes significantly in label-scarce environments. In contrast, unsupervised models like ContraMTD and Kitsune achieve relatively satisfactory performance without requiring labeled data. KerCL takes this further by leveraging unlabeled datasets with kernel-enhanced global dependency modeling and a self-supervised mechanism. For instance, KerCL achieves an 14.62% higher F1-score than MTSecurity on the TONIoT dataset, capitalizing on its ability to capture both local and global traffic patterns. Kitsune, however, focuses exclusively on individual packets, neglecting inter-relations, while ContraMTD's graph-based approach models primarily pairwise host interactions, overlooking broader network-wide dependencies. KerCL addresses these limitations by employing global kernel dependency modeling to capture comprehensive traffic interactions across the graph, providing deeper insights into network behavior.

Pretraining-based methods, indicating the promise of pre-training for EMTD. Among these, ET-BERT achieves competitive results by adopting transformer-based architectures for feature extraction. However, its reliance on complex architectures and large-scale models incurs high computational costs, limiting its versatility for more complex detection tasks. TrafCL, relying on heuristic augmentations, often disrupts the intrinsic semantic structure of traffic data, resulting in suboptimal representations. Flow-MAE and YaTC adopt MAE approaches to model traffic data, but both struggle to preserve semantic consistency and global dependencies, particularly under noisy or fragmented traffic scenarios. KerCL distinguishes itself by addressing these limitations through its dependency-guided graph reconstruction and dependency-aligned contrastive mechanism, ensuring semantically consistent augmented views and robust global representations.

C. Robustness of the Model (Q2)

To evaluate the robustness of the proposed KerCL framework, we conducted comprehensive experiments across

various noise levels and label-scarce conditions. Robustness in EMTD is critical due to the inherent noise and incomplete session information present in real-world network environments. We designed experiments focusing on two key aspects: effectiveness under limited labeled data and resistance to noise perturbation.

We first evaluated KerCL's performance under label-scarce scenarios by varying the number of labeled samples per class during training process. As shown in Figure 3, KerCL achieves superior performance compared to other pretraining-based models. With only 100 labeled samples per class, KerCL maintains high classification F1-score. This highlights KerCL's ability to capture the key semantic information in the traffic graph through its kernel-enhanced global dependency modeling, and contrastive learning based on semantic alignment further utilized the data effectively.

Next, we assessed KerCL's resistance to noise by introducing synthetic noise (randomly corrupting node features and edge connections) at increasing rates. Figure 4 illustrates performance trends of KerCL and baselines under increasing noise levels on DoHBrw dataset. Notably, KerCL consistently outperforms state-of-the-art baselines, maintaining high accuracy and F1-scores even when subjected to substantial data corruption. Additionally, after 100 training runs, KerCL exhibits significantly lower variance compared to other methods.

From a theoretical perspective, the robustness of KerCL arises from the following: 1) the HSIC-based alignment, which constrains the learned representations by maximizing mutual dependence between original and augmented views, preserving semantic consistency despite noisy augmentations; 2) the kernel enhanced global dependency modeling module, which captures global dependencies while minimizing the impact of local feature noise, providing an implicit smoothing effect that avoids over-smoothing issues in traditional GNNs; 3) and the combined contrastive and HSIC losses that reduce the generalization bound by ensuring diverse yet semantically aligned representations, thereby improving performance under both noise-free and noisy conditions.

Moreover, our HSIC-based regularization not only aligns augmented representations with their originals but also enhances robustness to input noise and perturbations. By

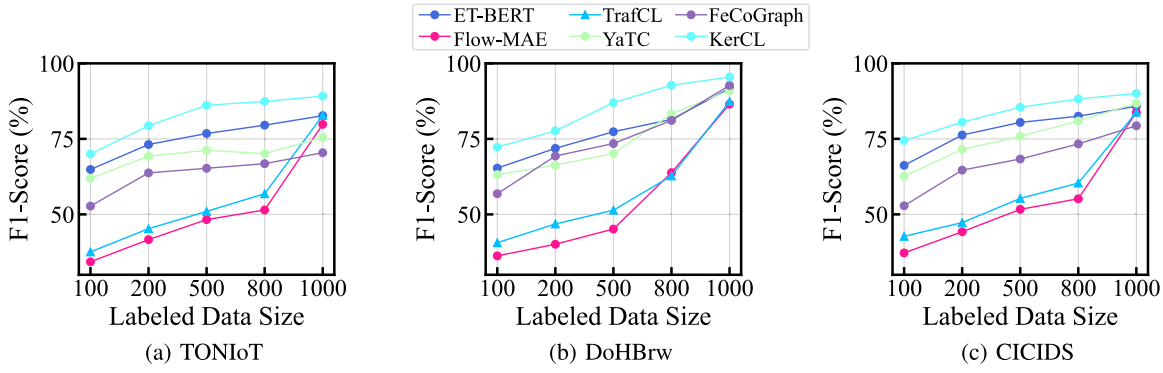


Fig. 3. Comparison of hidden dimensions and k similarity impacts on F1-Score.

TABLE III
ABLATION STUDY OF OUR KERCL

Method	KGC ¹	CL ¹	HSIC ¹	SS ¹	TONIoT		DoHBrw		CICIDS	
					AC	F1	AC	F1	AC	F1
(a) w/o $\mathcal{L}_s$	×	✓	×	✓	79.29 (± 1.56)	78.64 (± 1.50)	94.32 ($\pm .49$)	93.43 ($\pm .55$)	87.85 ($\pm .45$)	86.11 ($\pm .48$)
(b) w/o $\mathcal{L}_c$	✓	×	✓	✓	76.21 (± 1.88)	75.66 (± 1.88)	89.77 ($\pm .35$)	89.77 ($\pm .35$)	87.93 ($\pm .58$)	86.12 ($\pm .58$)
(c) w/o $\mathcal{L}_{CE}$	✓	✓	✓	×	76.60 (± 2.04)	76.21 (± 1.97)	94.63 ($\pm .57$)	93.19 ($\pm .60$)	87.83 ($\pm .32$)	86.23 ($\pm .30$)
(d) s-k ²	×	✓	✓	✓	79.10 (± 1.99)	79.67 (± 1.92)	93.08 (± 1.14)	93.07 (± 1.14)	86.58 ($\pm .55$)	86.58 ($\pm .55$)
(e) w/o $\mathcal{L}_s$ (s-k)	×	✓	×	✓	78.09 (± 2.14)	78.29 (± 2.24)	84.62 ($\pm .51$)	84.47 ($\pm .54$)	86.46 ($\pm .63$)	84.76 ($\pm .77$)
(f) w/o $\mathcal{L}_c$ (s-k)	×	×	×	✓	78.38 (± 2.22)	77.90 (± 2.20)	84.35 ($\pm .36$)	84.04 ($\pm .38$)	87.74 ($\pm .65$)	85.02 ($\pm .73$)
(g) w/o $\mathcal{L}_{CE}$ (s-k)	×	✓	×	×	77.19 (± 1.98)	76.56 (± 1.92)	81.85 (± 1.14)	81.73 (± 1.06)	86.10 ($\pm .75$)	84.41 ($\pm .79$)
Default	✓	✓	✓	✓	88.23 (± 0.22)	88.23 (± 0.22)	95.83 (± 0.36)	95.57 (± 0.37)	90.13 (± 0.21)	90.06 (± 0.21)

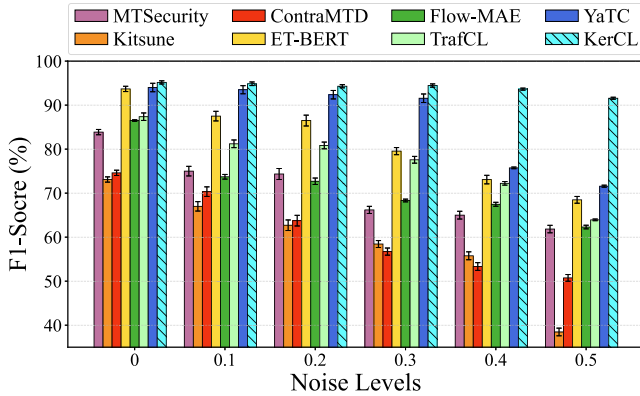
¹ KGC: Kernel-based Graph Convolutions. CL: Contrastive Learning. HSIC: Hilbert-Schmidt Independence Criterion. SS: Supervised Signal.² s-k: Strategy using kNN to construct augmented views.

Fig. 4. Different noise level study w.r.t. F1-score within the DoHBrw dataset (higher is better).

constraining kernel matrix variance, this regularization ensures embeddings are roughly uniformly distributed, retaining maximal data information while maintaining stability and uniformity in high-dimensional spaces [46].

D. Ablation Study (Q3)

In this section, we conduct a comprehensive ablation study to evaluate the contributions of each component in proposed KerCL, with results summarized in Table III. We structure our analysis around three key findings.

Synergy of CL and HSIC Alignment. The core of our self-supervised framework relies on the interplay between the instance-level contrastive loss ($\mathcal{L}_c$) and the global HSIC

alignment loss ($\mathcal{L}_s$). Removing either component leads to a significant performance drop. As shown in row (b), removing the contrastive loss causes the most severe degradation, with the F1-score on TONIoT plummeting by over 12% (from 88.23% to 75.66%). This highlights the critical role of the contrastive objective in learning discriminative features. Furthermore, removing the HSIC term (row (a)) also notably harms performance, indicating that instance-level discrimination is not enough; the global semantic consistency enforced by HSIC is crucial for robustness. The superior performance of the KerCL demonstrates a clear synergy: CL provides instance-level discrimination, while HSIC provides the global semantic regularization needed for robust representations.

Importance of the Supervised Signal. While KerCL is designed to learn effectively from unlabeled data, the supervised signal remains vital for achieving optimal performance. In row (c), we train the model without the cross-entropy loss ($\mathcal{L}_{CE}$). The performance drop is substantial across all datasets, confirming that the self-supervised objectives, while powerful for representation learning, are best utilized when fine-tuned with even a small amount of labeled data to align the learned features with the final classification task.

Effectiveness of Kernel-Based Graph Reconstruction. Finally, we validate our proposed dependency-guided graph reconstruction by comparing it against a standard kNN baseline (rows (d)-(g)). The full kNN-based variant ('s-k' in row (d)) underperforms our KerCL by a large margin across all metrics. For instance, on the CICIDS dataset, our default model achieves an F1-score of 90.06%, whereas the

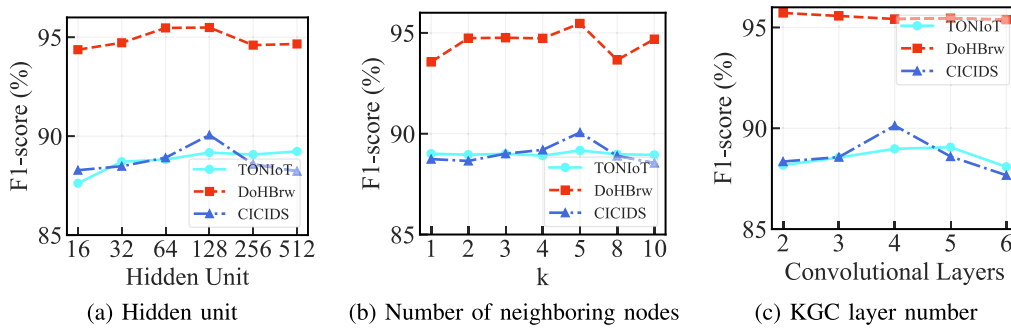


Fig. 5. F1-Score w.r.t. different parameters across three datasets.

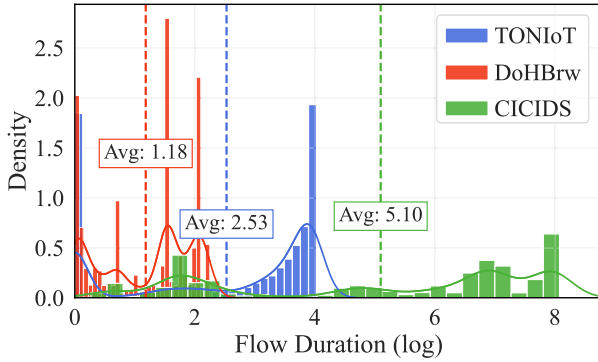


Fig. 6. Distribution density of traffic duration across three datasets (log-scaled).

kNN-based model only reaches 86.58%. This performance gap underscores the necessity of the global semantic information captured by our kernel-based modeling, which is more effective than the simpler, feature-based similarity used by kNN.

E. Parameter Sensitivity Analysis (Q4)

To investigate the impact of hyper-parameters and traffic data scale on KerCL's performance, we conduct experiments across the TONIOT, DoHBrw, and CICIDS datasets. Figure 6 shows the flow duration density distribution on these datasets. Among them, we log the data to eliminate the difference between the maximum and minimum values. Furthermore, it can be seen that the three datasets represent the distribution of different traffic in the real network respectively.

Impact of Network Dimensions. Figure 5(a) illustrates that increasing the dimensions of hidden units initially boosts performance. However, beyond a certain threshold, the performance stabilizes, yielding similar results across datasets.

Impact of the Number of Neighboring Nodes (k). Figure 5(b) examines the sensitivity of performance to the number of neighboring nodes (k) used for augmented views. The results reveal that the model achieves optimal performance across all three datasets at $k = 5$. Using fewer than 5 neighbors can result in information loss in the augmented views, whereas including more neighbors introduces noise and irrelevant information from unrelated nodes. Furthermore, larger k values reduce computational efficiency without significant performance gains, underscoring the importance of

TABLE IV
COMPLEXITY ANALYSIS OF THE PROPOSED FRAMEWORK

Steps	Time Complexity	Space Complexity
1	$\mathcal{O}(n^2d + l \cdot \mathcal{E} \cdot d)$	$\mathcal{O}(n^2 + nd + lnd + \mathcal{E})$
2	$\mathcal{O}(n^2 + kn + \mathcal{E})$	$\mathcal{O}(n) + \mathcal{V} + e $
3	$\mathcal{O}(b^2d^2)$	$\mathcal{O}(bd)$
Total	$\mathcal{O}(n^2d)$	$\mathcal{O}(n^2)$

¹ Refers to kernel-based graph convolutions.

² Refers to dependency-guided graph reconstruction.

³ Refers to HSIC-based semantic alignment.

carefully selecting k to balance representation quality and computational cost.

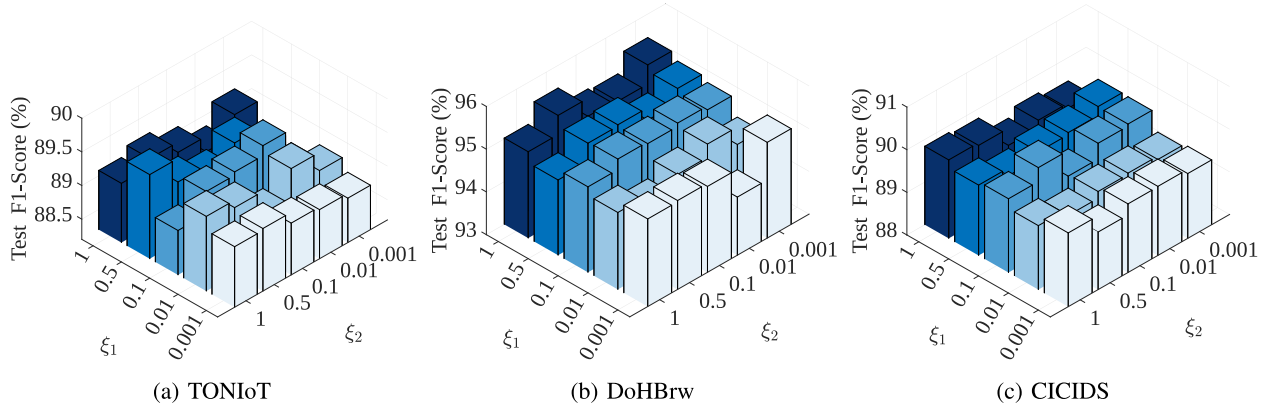
Impact of KGC Layer Number. Figure 5(c) investigates the effect of kernel propagation layer number on F1-score. It shows that deeper networks exhibit more stable performance compared to deep GCNs. As the layer number increases, KGC's performance initially improves and then slightly decreases. Notably, KGC with a fixed number of layers performs better, showing less performance degradation and maintaining greater stability.

Impact of the Importance Parameters ξ in $\mathcal{L}_s$. We evaluated the performance of the KerCL under different combinations of importance parameters across 25 different view embeddings. Specifically, we adjusted the trade-off parameters ξ_1 and ξ_2 within the range of $\{0.001, 0.01, 0.1, 0.5, 1\}$. The results of the sensitivity analysis are shown in Figure 7. The figure shows performance fluctuations below 1.5%.

On the whole, our method is insensitive to these parameters on different datasets, and the performance difference of model is not significant as long as they are within a reasonable range.

F. Analysis of Scalability and Overhead

Analysis of Scalability. We analyze the complexity of our KerCL in terms of both time complexity and space complexity, as summarized in Table IV. Here, n is the number of nodes, d is the feature dimension, l is the number of KGC layers, $|\mathcal{E}|$ denotes the number of edges. Let b denote the number of nodes in a batch, k is the number of neighboring nodes, and e represent the number of neighborhood nodes per node during the reconstruction process. The framework consists of

Fig. 7. Impact of importance parameters (ξ_1, ξ_2) on F1-Score across three datasets.TABLE V
HYPER-PARAMETER CONFIGURATIONS FOR KERCL

Notation	Description	Value
lr	Learning rate	0.01
hid	Number of hidden units	128
$L2$	Weight decay of optimizer	5×10^{-4}
$[\xi_1, \xi_2]$	Trade-off parameters	$[0.1, 0.01]$
e	Number of neighborhood nodes	5
λ_c	Weight of contrastive loss term	0.1
λ_s	Weight of kernel dependence term	$\{0.5, 1, 0.1\}^1$
λ_e	Weight of augmented view classification loss	0.1

¹ Values correspond to three different datasets.

three major steps, i) kernel-based graph convolutions, ii) graph reconstruction, and iii) semantic alignment. It is worth noting that the kernel matrix generation step (i.e., the first step) is executed only once and reused during the training process. Consequently, its quadratic time complexity ($\mathcal{O}(n^2d)$) is computationally feasible for large graphs despite n .

Analysis of Hyper-Parameter Configurations. We demonstrate the hyper-parameter configuration used in our experiment. Table V summarizes the key hyper-parameters, their descriptions, and values employed. These hyper-parameters are carefully selected to balance model performance, convergence stability, and computational efficiency.

G. Case Study

Figure 8 shows the confusion matrix for the CICIDS dataset and allows a fine-grained view of class-level performance. Most entries lie on the main diagonal, confirming the model's high overall accuracy. Two security-critical findings stand out. First, false alarms are rare: only a handful of benign flows were tagged as attacks. Second, missed detections are equally limited: very few malicious flows slipped through as benign. The remaining errors arise mainly from confusions among related attack types, for example between different denial-of-service or web-based attacks, rather than between benign and malicious traffic. These observations demonstrate that KerCL fulfils the primary requirement of reliably separating malicious activity from normal traffic.

		Confusion Matrix													
True Label	BENIGN	395	30	0	27	0	0	0	0	0	0	0	0	0	0
	DoS GoldenEye	16	254	0	10	8	0	0	0	0	0	0	0	0	0
	PortScan	0	0	475	0	0	0	0	0	0	0	0	0	0	0
	DoS Slowhttptest	15	13	0	273	20	0	0	0	0	0	0	0	0	0
	Brute Force	0	20	0	26	273	0	0	0	0	0	0	0	0	0
	Bot	0	0	0	0	10	292	0	0	0	0	0	0	0	0
	SQL Injection	0	0	0	0	0	1	1	1	0	0	0	0	0	0
	XSS	8	10	0	10	20	0	30	0	50	0	0	0	0	0
	Infiltration	0	0	0	0	0	0	0	7	0	0	0	0	0	0
	DDoS	0	21	0	0	0	10	0	0	417	0	0	0	0	0
	DoS slowloris	0	18	0	0	0	0	0	0	15	278	0	0	0	0
	Heartbleed	0	0	0	0	0	0	0	0	0	0	1	0	0	0
	FTP-Patator	0	0	0	0	0	0	0	0	0	0	292	1	0	0
	DoS Hulk	0	0	0	0	30	0	0	0	0	38	0	496	0	0
	SSH-Patator	0	0	0	0	0	0	0	0	0	0	0	0	288	0
		Predicted Label													

Fig. 8. The confusion matrix for the CICIDS dataset.

To illustrate how the learned representations support this behaviour, we use t-SNE to visualize the embedding of benign samples and malicious samples across three datasets. Figure 9 and Figure 10 provide insights into the capability of these methods to learn differentiated embedding.

Figure 9 plots the embeddings produced by KerCL together with the decision regions of a linear SVM. Benign flows (blue) and malicious flows (gray) form well-separated clusters across TONIoT, DoHBrw, and CICIDS, yielding accuracies of 96.6%, 97.5%, and 94.8%, respectively. These clear margins confirm that KerCL captures both local and global dependencies and remains robust under noise and label scarcity.

In contrast, Figure 10 visualizes the embeddings and decision accuracy achieved by YaTC on the same datasets. While YaTC also achieves competitive accuracy (e.g., 95.2% on TONIoT and 95.2% on DoHBrw), there is more overlap between benign and malicious traffic clusters, particularly on CICIDS dataset. For instance, on CICIDS, YaTC achieves an accuracy of only 90.7%, which is significantly lower than KerCL's 94.8%. This overlap indicates that YaTC struggles to learn the hidden, differentiated contextual information needed to separate traffic classes effectively, likely due to its limited

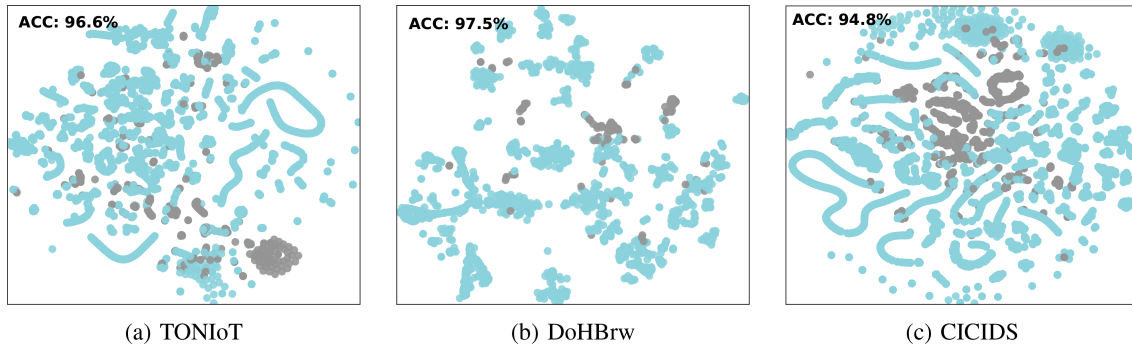


Fig. 9. Embedding visualizations of KerCL on three datasets.

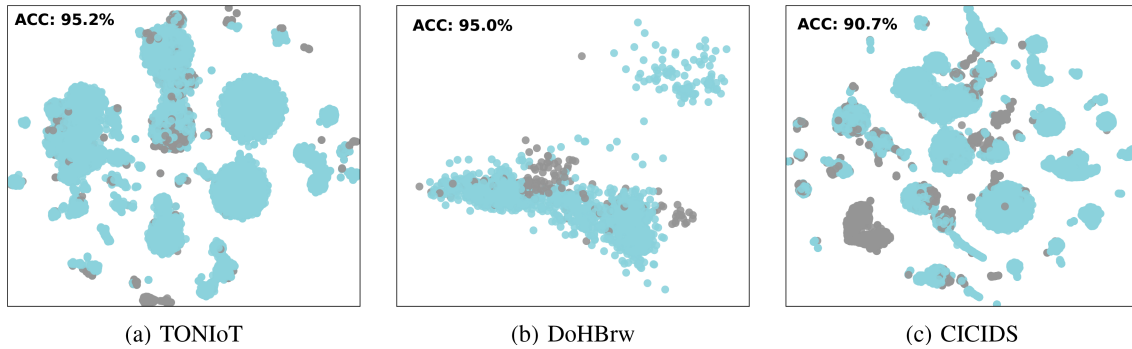


Fig. 10. Embedding visualizations of YaTC on three datasets.

ability to capture global dependencies and semantic consistency across augmented views.

V. CONCLUSION AND FUTURE WORK

In this work, we propose KerCL, a novel kernel-enhanced contrastive learning framework tailored for EMTD. By integrating global dependency modeling, kernel-based graph reconstruction, and HSIC-guided semantic alignment, KerCL effectively addresses challenges posed by limited labels, insufficient structural context, and noise sensitivity in graph-based learning. Experimental evaluations on three datasets demonstrate that KerCL outperforms state-of-the-art methods, particularly in noisy environments and scenarios with limited labeled data. Our future work will focus on reducing this complexity. Furthermore, we plan to validate our method's robustness against more sophisticated adaptive threats and extend its applicability by incorporating temporal dynamics and protocol-agnostic modeling for emerging traffic patterns.

REFERENCES

- [1] Y. Yue, X. Chen, Z. Han, X. Zeng, and Y. Zhu, "Contrastive learning enhanced intrusion detection," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 4, pp. 4232–4247, Dec. 2022.
- [2] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018.
- [3] X. Zhang, W. Geng, Y. Song, H. Cheng, K. Xu, and Q. Li, "Privacy-preserving and lightweight verification of deep packet inspection in clouds," *IEEE/ACM Trans. Netw.*, vol. 32, no. 1, pp. 159–174, Feb. 2024.
- [4] J. Fan, C. Guan, K. Ren, Y. Cui, and C. Qiao, "SPABox: Safeguarding privacy during deep packet inspection at a MiddleBox," *IEEE/ACM Trans. Netw.*, vol. 25, no. 6, pp. 3753–3766, Dec. 2017.
- [5] C. Fu, Q. Li, and K. Xu, "Flow interaction graph analysis: Unknown encrypted malicious traffic detection," *IEEE/ACM Trans. Netw.*, vol. 32, no. 4, pp. 2972–2987, Aug. 2024.
- [6] W. Li, X.-Y. Zhang, H. Bao, H. Shi, and Q. Wang, "ProGraph: Robust network traffic identification with graph propagation," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1385–1399, Jun. 2023.
- [7] J. Yang, X. Jiang, Y. Lei, W. Liang, Z. Ma, and S. Li, "MTSecurity: Privacy-preserving malicious traffic classification using graph neural network and transformer," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 3, pp. 3583–3597, Jun. 2024.
- [8] Z. Cheng et al., "Information diffusion prediction via meta-knowledge learners," *Inf. Sci.*, vol. 707, Jul. 2025, Art. no. 122034.
- [9] Z. Cheng, Y. Liu, T. Zhong, K. Zhang, F. Zhou, and P. S. Yu, "Disentangling inter- and intra-cascades dynamics for information diffusion prediction," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 8, pp. 4548–4563, Aug. 2025.
- [10] Q. Yuan, G. Gou, Y. Zhu, Y. Zhu, G. Xiong, and Y. Wang, "MCR: A unified framework for handling malicious traffic with noise labels based on multidimensional constraint representation," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 133–147, 2024.
- [11] C. Liu, B. Li, J. Zhao, Z. Zhen, X. Liu, and Q. Zhang, "FewM-HGCL: Few-shot malware variants detection via heterogeneous graph contrastive learning," *IEEE Trans. Dependable Secure Comput.*, early access, Oct. 25, 2022, doi: [10.1109/TDSC.2022.3216902](https://doi.org/10.1109/TDSC.2022.3216902).
- [12] X. Han et al., "ContraMTD: An unsupervised malicious network traffic detection method based on contrastive learning," in *Proc. Int. World Wide Web Conf. (WWW)*, pp. 1680–1689, 2024, pp. 1680–1689.
- [13] S. Suresh, P. Li, C. Hao, and J. Neville, "Adversarial graph augmentation to improve graph contrastive learning," in *Proc. Adv. Neural Inf. Process. Syst. (Neurips)*, 2021, pp. 15920–15933.
- [14] Y. Min, F. Wenkel, and G. Wolf, "Scattering GCN: Overcoming over-smoothness in graph convolutional networks," in *Proc. Adv. Neural Inf. Process. Syst. (Neurips)*, 2020, pp. 14498–14508.
- [15] R. Yang, W. Dai, C. Li, J. Zou, and H. Xiong, "Tackling over-smoothing in graph convolutional networks with EM-based joint topology optimization and node classification," *IEEE Trans. Signal Inf. Process. Over Netw.*, vol. 9, pp. 123–139, 2023.
- [16] M. Lin, T. Xiao, E. Dai, X. Zhang, and S. Wang, "Certifiably robust graph contrastive learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 17008–17037.

- [17] Q. Mao et al., “FeCoGraph: Label-aware federated graph contrastive learning for few-shot network intrusion detection,” *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 2266–2280, 2025.
- [18] J. Wen, Z. Zhang, Y. Xu, and Z. Zhong, “Incomplete multi-view clustering via graph regularized matrix factorization,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2019, pp. 593–608.
- [19] X. Pu, S. L. Chau, X. Dong, and D. Sejdinovic, “Kernel-based graph learning from smooth signals: A functional viewpoint,” *IEEE Trans. Signal Inf. Process. Over Netw.*, vol. 7, pp. 192–207, 2021.
- [20] X. Liu et al., “Mapping the unseen: Robust IP geolocation through the lens of uncertainty quantification,” *Comput. Netw.*, vol. 269, Sep. 2025, Art. no. 111405.
- [21] W. Mayer and M. Schmiedecker, “TLScmpare: Crowdsourcing rules for HTTPS everywhere,” in *Proc. 25th Int. Conf. Companion World Wide Web - WWW Companion*, 2016, pp. 471–476.
- [22] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 785–794.
- [23] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, “Robust smartphone app identification via encrypted network traffic analysis,” *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 1, pp. 63–78, Jan. 2018.
- [24] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, “Accurate decentralized application identification via encrypted traffic analysis using graph neural networks,” *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.
- [25] Y. Liu, X. Wang, B. Qu, and F. Zhao, “ATVITSC: A novel encrypted traffic classification method based on deep learning,” *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 9374–9389, 2024.
- [26] P. Sirinam, M. Imani, M. Juarez, and M. Wright, “Deep fingerprinting: Undermining website fingerprinting defenses with deep learning,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Toronto, ON, Canada, Oct. 2018, pp. 1928–1943.
- [27] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, “FS-Net: A flow sequence network for encrypted traffic classification,” in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2019, pp. 1171–1179.
- [28] H. Xu, S. Li, Z. Cheng, R. Qin, J. Xie, and P. Sun, “TrafficGCN: Mobile application encrypted traffic classification based on GCN,” in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2022, pp. 891–896.
- [29] Z. Fu et al., “Encrypted malware traffic detection via graph-based network analysis,” in *Proc. Int. Symp. Res. Attacks, Intrusions Defenses*, Oct. 2023, pp. 297–314.
- [30] Z. Diao et al., “EC-GCN: A encrypted traffic classification framework based on multi-scale graph convolution networks,” *Comput. Netw.*, vol. 224, Apr. 2023, Art. no. 109614.
- [31] L. Liu, T. Sun, L. Yang, and J. Bai, “TAM-CNN: An enhanced malicious encrypted traffic detection method with feature-threshold images,” in *Proc. ACM Turing Award Celebration Conf. (ACM-TURC)*, 2024, pp. 171–176.
- [32] X. Yang, S. Ruan, J. Li, Y. Yue, and B. Sun, “TrafCL: Robust encrypted malicious traffic detection via contrastive learning,” in *Proc. ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2024, pp. 2910–2919.
- [33] K. Xu et al., “Self-supervised learning malware traffic classification based on masked autoencoder,” *IEEE Internet Things J.*, vol. 11, no. 10, pp. 17330–17340, May 2024.
- [34] C. Wu, J. Sun, J. Chen, M. Alazab, Y. Liu, and Y. Xiang, “TCG-IDS: Robust network intrusion detection via temporal contrastive graph learning,” *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 1475–1486, 2025.
- [35] J. Sun, X. Zhang, Y. Wang, and S. Jin, “CoMDet: A contrastive multi-modal pre-training approach to encrypted malicious traffic detection,” in *Proc. IEEE Annu. Comput., Softw., Appl. Conf. (COMPSAC)*, Jul. 2024, pp. 1118–1125.
- [36] G. Wu, X. Wang, and J. Zhang, “PeerG: A P2P botnet detection method based on representation learning and graph contrastive learning,” *Comput. Secur.*, vol. 140, May 2024, Art. no. 103775.
- [37] X. Han, S. Liu, J. Liu, B. Jiang, Z. Lu, and B. Liu, “ECNet: Robust malicious network traffic detection with multi-view feature and confidence mechanism,” *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 6871–6885, 2024.
- [38] A. Gretton, O. Bousquet, A. Smola, and B. Schölkopf, “Measuring statistical dependence with Hilbert–Schmidt norms,” in *Algorithmic Learning Theory (Lecture Notes in Computer Science)*, vol. 3734, S. Jain, H. U. Simon, and E. Tomita, Eds., Springer, 2005, pp. 63–77.
- [39] N. Moustafa, “A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets,” *Sustain. Cities Soc.*, vol. 72, Sep. 2021, Art. no. 102994.
- [40] M. MontazeriShatoori, L. Davidson, G. Kaur, and A. Habibi Lashkari, “Detection of DoH tunnels using time-series classification of encrypted traffic,” in *Proc. IEEE Int. Conf. Dependable, Automatic Secure Comput., Int. Conf. Pervasive Intell. Comput., Int. Conf. Cloud Big Data Comput., Int. Conf. Cyber Sci. Technol. Congr. (DASC/PiCom/CBDCCom/CyberSciTech)*, Aug. 2020, pp. 63–70.
- [41] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2018, pp. 108–116.
- [42] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, “Characterization of encrypted and VPN traffic using time-related,” in *Proc. 2nd Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2016, pp. 407–414.
- [43] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, “ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification,” in *Proc. Int. World Wide Web Conf. (WWW)*, pp. 633–642, 2022, 2022, pp. 633–642.
- [44] Z. Hang, Y. Lu, Y. Wang, and Y. Xie, “Flow-MAE: Leveraging masked AutoEncoder for accurate, efficient and robust malicious traffic classification,” in *Proc. 26th Int. Symp. Res. Attacks, Intrusions Defenses*, Oct. 2023, pp. 297–314.
- [45] R. Zhao et al., “A novel self-supervised framework based on masked autoencoder for traffic classification,” *IEEE/ACM Trans. Netw.*, vol. 32, no. 3, pp. 2012–2025, Jun. 2024.
- [46] T. Wang and P. Isola, “Understanding contrastive representation learning through alignment and uniformity on the hypersphere,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 9929–9939.

Meihui Zhong received the M.S. degree from the College of Computer and Cyber Security, Fujian Normal University, in 2024. She is currently pursuing the Ph.D. degree with the University of Electronic Science and Technology of China (UESTC). Her research interests include encrypted traffic analysis and trustworthy artificial intelligence.

Ting Zhong (Member, IEEE) received the Ph.D. degree in information and communication engineering from the University of Electronic Science and Technology of China (UESTC) in 2009. She is currently a Full Professor with the School of Information and Software Engineering, UESTC. Her current research interests include deep learning, social networks, and cloud computing.

Yong Wang is currently a Post-Doctoral Fellow at The Hong Kong University of Science and Technology (HKUST). His research interests include real-time IP address-based geolocation, internet measurement, and analysis. He is a member of China Computer Federation (CCF).

Kai Chen (Senior Member, IEEE) received the Ph.D. degree in computer science from Northwestern University in 2012. He joined HKUST in July 2012, where he directs the Intelligent System and Networking (iSING) Laboratory and the WeChat-HKUST Joint Lab on Artificial Intelligence Technology (WHAT LAB). His research interests include AI-centric networking and machine learning systems.

Fan Zhou (Member, IEEE) received the Ph.D. degree from the University of Electronic Science and Technology of China (UESTC) in 2011. He is currently a Full Professor with the School of Information and Software Engineering, UESTC. His research interests include machine learning and social network knowledge discovery.